

## 4. CCS: Calculus of Communicating Systems (parallel)

---

José Proença

*(slides mainly from Nelma Moreira)*

Concurrent programming (CC3040) 2025/2026

CISTER – U.Porto, Porto, Portugal

<https://fm-dcc.github.io/cp2526>



- "." **Prefixo** a execução de  $\alpha.P$  começa com a execução da ação  $\alpha \in Act$  e depois comporta-se como  $P$
- "+" **Escolha** O processo  $P + Q$  comporta-se como o processo  $P$  ou o processo  $Q$ .  
É a escolha não determinística
- "|" **Composição Paralela** O processo  $P|Q$  representa a execução concorrente de  $P$  e  $Q$  (que progridem independentemente no tempo).

## Exemplo

*Exemplos de 1 – Buffer (de uma posição)*

*Considera as equações de  $\Gamma$  seguintes.*

1. Calcula  $\llbracket \text{Buffer} \rrbracket_{\Gamma}$ .

$$\text{Buffer} := \text{put?}.\text{get?}.\text{Buffer}$$

2. Calcula  $\llbracket \text{put?}.\text{BufferM} \rrbracket_{\Gamma}$ .

$$\text{BufferM} := \text{put?}.\text{get?}.\text{BufferM} + \text{get?}.\text{put?}.\text{BufferM}$$

3. Calcula  $\llbracket \text{Buffer0} \rrbracket_{\Gamma}$ .

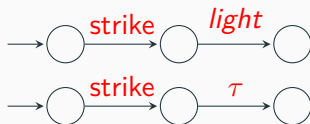
$$\text{Buffer0} := \text{put?}.\text{Buffer1}$$

$$\text{Buffer1} := \text{get?}.\text{Buffer2} + \text{get?}.\text{Buffer0}$$

$$\text{Buffer2} := \text{get?}.\text{Buffer1}$$

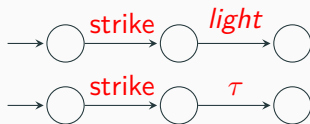
Vimos que a composição paralela (intercalada) de dois sistemas de transição correspondia ao seu produto.

Por exemplo, tendo dois fósforos: Um acende, outro não

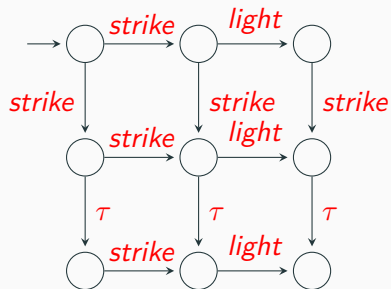


Vimos que a composição paralela (intercalada) de dois sistemas de transição correspondia ao seu produto.

Por exemplo, tendo dois fósforos: Um acende, outro não



O produto seria então:

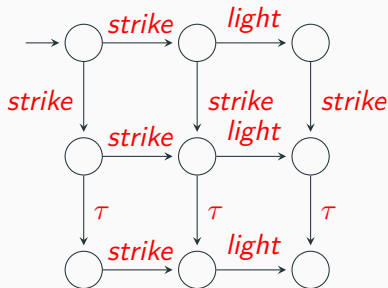


Vimos que a composição paralela (intercalada) de dois sistemas de transição correspondia ao seu produto.

Podíamos então ter o seguinte termo do  $CCS_0$ :

$$\textit{strike.light.strike}.\tau.0 + \textit{strike.strike.light}.\tau.0 + \\ \textit{strike}.\tau.\textit{strike.light}.0 + \textit{strike.strike}.\tau.\textit{light}.0$$

O produto seria então:



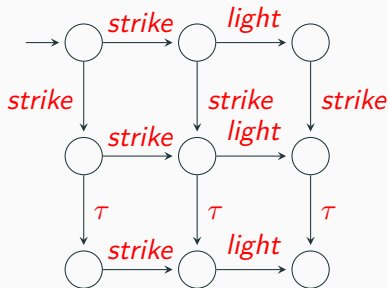
Vimos que a composição paralela (intercalada) de dois sistemas de transição correspondia ao seu produto.

Podíamos então ter o seguinte termo do  $CCS_0$ :

$$strike.light.strike.\tau.0 + strike.strike.light.\tau.0 + \\ strike.\tau.strike.light.0 + strike.strike.\tau.light.0$$

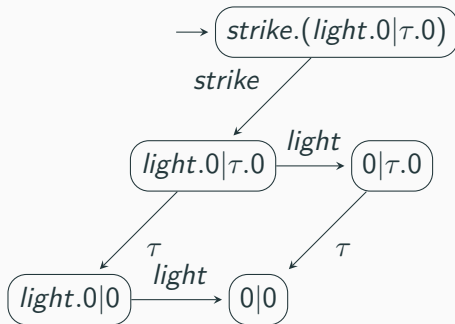
Mas vamos considerar um operador próprio que permite mais flexibilidade e termos mais concisos. No caso anterior seria:

O produto seria então:

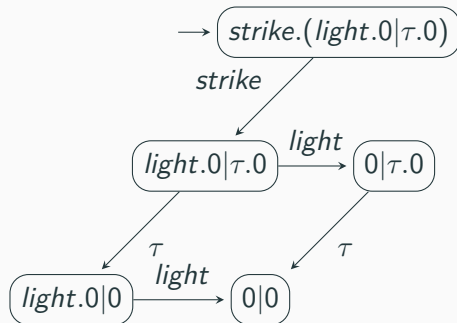


Mais dois fósforos:  $\text{strike.}(light.0|\tau.0)$

Mais dois fósforos:  $strike.(light.0|\tau.0)$



Mais dois fósforos:  $strike.(light.0|\tau.0)$



Supomos sempre que todas as ações são instantâneas.

$$\text{ParE} \frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q}$$

$$\text{ParD} \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}$$

Para que dois sistemas possam comunicar temos de implementar algum tipo de sincronia, neste caso de *handshaking*: dois processos comunicam com um par de ações que partilham o mesmo "nome".

Para que dois sistemas possam comunicar temos de implementar algum tipo de sincronia, neste caso de *handshaking*: dois processos comunicam com um par de ações que partilham o mesmo "nome". Seja

- O conjunto de ações observáveis  $Com$  é dividido em dois
- $Com = A^! \cup A^?$
- $A^!$  conjunto de ações de saída (*output*) (envio)
- $A^?$  conjunto de ações de entrada (*input*) (recebidas)
- Ações com o mesmo nome formam um par e são complementares:
- um processo envia  $a^!$  e o outro recebe  $a^?$
- O complemento de  $a \in A^! \cup A^?$  designa-se por  $\bar{a}$ : se  $a \in A^!$  então  $\bar{a} \in A^?$  e vice-versa.
- Para a ação interna  $\tau$ , temos  $\tau = \bar{\tau}$ .
- $\forall \alpha \in Act, \bar{\bar{\alpha}} = \alpha$ .

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\quad} P'|Q'}$$

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

A sincronização não é observável por processos externos.

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

A sincronização não é observável por processos externos.

A ação interna  $\tau$  representa a sincronização para o exterior.

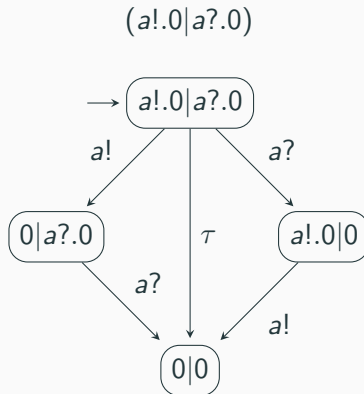
$$\text{Sync} \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

---

$$\text{ParE} \frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q}$$

$$\text{ParD} \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}$$

$(a!.0|a?.0)$



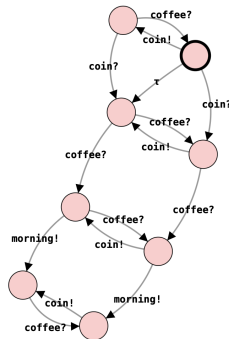
A regra Sync não exclui a utilização de ParD e ParE.

Seja o seguinte modelo de uma máquina de café e um utilizador:

$$User := coin!.coffee?.morning!.0$$

$$Machine := coin?.coffee!.Machine$$

Desenha o LTS de  $\llbracket (User|Machine) \rrbracket_{\tau}$   
usando as regras de inferência.



- Proíbe que pares de ações observáveis ( $a$  e  $\bar{a}$ ) sejam usadas individualmente.
- Força a sincronia (a aplicação da regra Sync)
- $P \setminus H$  onde  $P$  é um processo e  $H$  um conjunto de ações de comunicação que serão proibidas (por vezes em vez de  $a!$ ,  $a?$  apenas se escreve  $a$  em  $H$ ).
- As ações internas  $\tau$  não podem estar em  $H$

$$\text{Res} \frac{P \xrightarrow{\alpha} P' \quad \alpha \notin H}{P \setminus H \xrightarrow{\alpha} P' \setminus H}$$

$$((a!.0|a!.0)|a?.0)\backslash\{a!, a?\} \xrightarrow{\tau} ((a!.0|0)|0)\backslash\{a!, a?\}$$

$$((a!.0|a!.0)|a?.0)\backslash\{a!, a?\} \xrightarrow{\tau} ((0|a!.0)|0)\backslash\{a!, a?\}$$


---

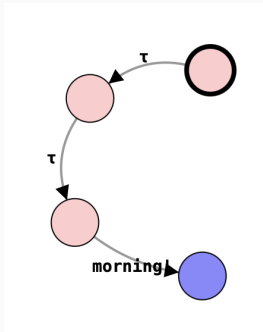
$$\text{ParE} \frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q}$$

$$\text{ParD} \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}$$

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

$$\text{Res} \frac{P \xrightarrow{\alpha} P' \quad \alpha \notin H}{P \backslash H \xrightarrow{\alpha} P' \backslash H}$$

No exemplo anterior calcula o LTS  $\llbracket (User|Machine) \setminus \{coin, coffee\} \rrbracket_{\Gamma}$  e testa no Pseuco.com.



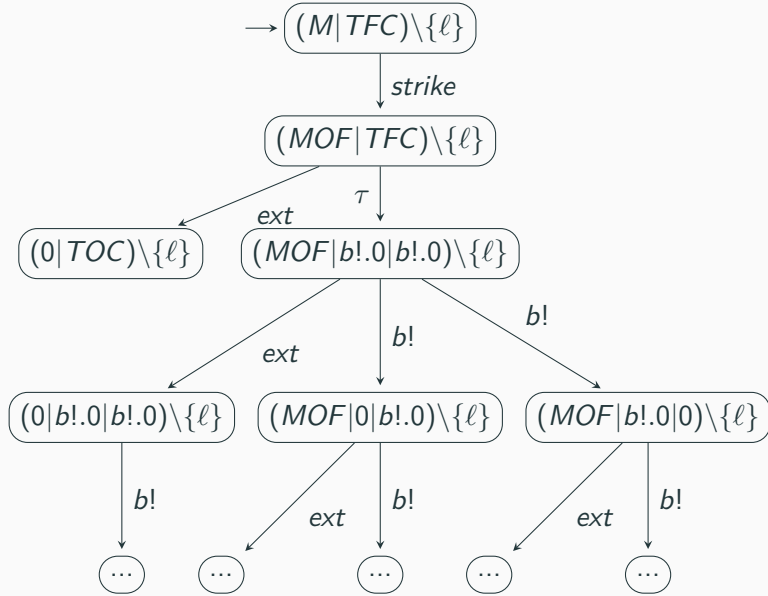
Indica o termo que etiqueta cada estado.

Considera as seguintes equações

$$\begin{aligned} Match &:= strike.MatchOnFire \\ MatchOnFire &:= light!.MatchOnFire + extinguish!.0 \\ TwoFireCracker &:= light?.(bang!.0|bang!.0) \end{aligned}$$

Constrói o LTS a expressão abaixo e testa no Pseuco.com.

$$(Match|TwoFireCracker)\{light\}$$



Seja  $Com = A^! \cup A^?$  conjunto de ações de comunicação,  $Act = Com \cup \{\tau\}$  um conjunto de ações, e  $Var$  um conjunto de nomes (variáveis). As expressões do CCS são

$$P ::= 0 \mid X \mid P + P \mid \alpha.P \mid P|P \mid P \setminus H$$

onde  $\alpha \in Act$ ,  $X \in Var$  e  $H \subseteq Com$ . Supomos um conjunto  $\Gamma$  de equações  $X := P$ .

A semântica das expressões do CCS é então

$$\llbracket \_ \rrbracket : (Var \rightarrow CCS) \rightarrow CCS \rightarrow LTS_{CCS}$$

tal que

$$\llbracket P \rrbracket_{\Gamma} = (CCS, \longrightarrow_{\Gamma}, P)$$

com

$$LTS_{CCS} = \{(CCS, T, s) \mid T \subseteq CCS \times Act \times CCS, \wedge s \in CCS\}$$

onde  $\longrightarrow_{\Gamma}$  a mais pequena relação que satisfaz as regras de inferência.

$$\begin{array}{llll}
 \text{Pref} \frac{}{\alpha.P \xrightarrow{\alpha} P} & \text{EscD} \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'} & \text{Sync} \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'} & \text{Res} \frac{P \xrightarrow{\alpha} P' \quad \alpha \notin H}{P \setminus H \xrightarrow{\alpha} P' \setminus H} \\
 \text{ParE} \frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q} & \text{EscE} \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'} & \text{ParD} \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'} & \text{Rec} \frac{P \xrightarrow{\alpha} P' \quad \Gamma(X) = P}{X \xrightarrow{\alpha} P'}
 \end{array}$$

- $P|Q|R$  é  $(P|Q)|R$

- $P|Q|R$  é  $(P|Q)|R$
- $\alpha.P|Q$  é  $(\alpha.P)|Q$

- $P|Q|R$  é  $(P|Q)|R$
- $\alpha.P|Q$  é  $(\alpha.P)|Q$
- $P + R|Q$  é  $(P + R)|Q$

- $CCS_0$  só pode produzir sistemas de transição finitos acíclicos

- $CCS_0$  só pode produzir sistemas de transição finitos acíclicos
- $CCS_0^\omega$  com  $\Gamma$  finito só pode produzir sistemas de transição estado-finitos (a menos de isomorfismo)

- $CCS_0$  só pode produzir sistemas de transição finitos acíclicos
- $CCS_0^\omega$  com  $\Gamma$  finito só pode produzir sistemas de transição estado-finitos (a menos de isomorfismo)
- $CCS$  pode produzir sistemas e transição infinitos e com ramificação infinita

- $CCS_0$  só pode produzir sistemas de transição finitos acíclicos
- $CCS_0^\omega$  com  $\Gamma$  finito só pode produzir sistemas de transição estado-finitos (a menos de isomorfismo)
- CCS pode produzir sistemas e transição infinitos e com ramificação infinita
- Ex:  $X := a.0|X$  tem ramificação infinita (Verifica!)

- $CCS_0$  só pode produzir sistemas de transição finitos acíclicos
- $CCS_0^\omega$  com  $\Gamma$  finito só pode produzir sistemas de transição estado-finitos (a menos de isomorfismo)
- CCS pode produzir sistemas e transição infinitos e com ramificação infinita
- Ex:  $X := a.0|X$  tem ramificação infinita (Verifica!)
- com mais uma operação - renomeação  $P[f]$  tem a potência duma Máquina de Turing.

- Já vimos uma máquina de café em CCS

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?
- Seria igual apenas mudando a accção de envio.

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?
- Seria igual apenas mudando a accção de envio.
- $ChM := coin?.choco!.ChM$

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?
- Seria igual apenas mudando a accção de envio.
- $ChM := coin?.choco!.ChM$
- e o mesmo para outros produtos.

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?
- Seria igual apenas mudando a accção de envio.
- $ChM := coin?.choco!.ChM$
- e o mesmo para outros produtos.
- Então podemos ter

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?
- Seria igual apenas mudando a accção de envio.
- $ChM := coin?.choco!.ChM$
- e o mesmo para outros produtos.
- Então podemos ter
- $VM := coin?.item!.VM$

- Já vimos uma máquina de café em CCS
- $CM := coin?.coffee!.CM$
- Mas agora se quisermos um chocolate?
- Seria igual apenas mudando a accção de envio.
- $ChM := coin?.choco!.ChM$
- e o mesmo para outros produtos.
- Então podemos ter
- $VM := coin?.item!.VM$
- e definir

$$CM := VM[coffee/item]$$
$$ChM := VM[choc/item]$$

...

Seja  $f : Act \rightarrow Act$  uma função de renomeação tal que

$$\begin{aligned} f(\tau) &:= \tau, \\ f(\bar{a}) &:= \overline{f(a)} \quad \forall a \in Com. \end{aligned}$$

A função  $f$  pode representar-se da forma  $[b_1/a_1, \dots, b_n/a_n]$  se  $f(a_i) = b_i$ .

Então, sendo  $P$  um processo  $P[f]$  é também um processo em que cada ação  $a$  é substituída por  $f(a)$  (assim como os complementos).

$$\begin{aligned} (a.B + b.B)[a/b, b/a] &= (b.B + a.B) \\ (a.0 + \bar{a}.A)[a/b] &= (a.0 + \bar{a}.A) \end{aligned}$$

$$\text{Rel} \frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f(\alpha)} P'[f]}$$

$$\text{Rel} \frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f(\alpha)} P'[f]}$$

## Exemplo

Seja  $A := a.b.B$ , calcular

$$\llbracket (A|b.a.B) + (b.A)[a/b] \rrbracket_{\Gamma}$$

- Operadores dinâmicos:  $\cdot$  e  $+$

- Operadores dinâmicos:  $\cdot$  e  $+$
- Desaparecem após se efectuar uma transição

- Operadores dinâmicos:  $\cdot$  e  $+$
- Desaparecem após se efectuar uma transição

$$\text{Pref} \frac{}{\alpha.P \xrightarrow{\alpha} P}$$

$$\text{EscD} \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}$$

- Operadores estáticos:  $|$  e  $\backslash$  (também  $[f]$ )

- Operadores dinâmicos:  $\cdot$  e  $+$
- Desaparecem após se efectuar uma transição
- Operadores estáticos:  $|$  e  $\backslash$  (também  $[f]$ )
- Não desaparecem após ser efectuada uma transição

- Operadores dinâmicos:  $\cdot$  e  $+$
- Desaparecem após se efectuar uma transição
- Operadores estáticos:  $|$  e  $\backslash$  (também  $[f]$ )
- Não desaparecem após ser efectuada uma transição

$$\text{Sync} \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

$$\text{ParD} \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}$$

$$\text{Res} \frac{P \xrightarrow{\alpha} P' \quad \alpha \notin H}{P \backslash H \xrightarrow{\alpha} P' \backslash H}$$

Não é permitida recursão sobre operadores estáticos.

$$P ::= 0 \mid X \mid P + P \mid \alpha.P \mid R$$

$$R ::= 0 \mid R + R \mid \alpha.R \mid R|R \mid R \setminus H$$

Não é permitida recursão sobre operadores estáticos.

$$P ::= 0 \mid X \mid P + P \mid \alpha.P \mid R$$

$$R ::= 0 \mid R + R \mid \alpha.R \mid R|R \mid R \setminus H$$

### Proposição

*Se  $\Gamma$  é uma função parcial cujo contradomínio só tem expressões regulares então o  $\text{Reach}(\llbracket P \rrbracket_{\Gamma})$  é finito por estados para todo  $P \in \text{CCS}$  (i.e. o LTS atingível é finito por estados).*

Não é permitida recursão sobre operadores estáticos.

$$P ::= 0 \mid X \mid P + P \mid \alpha.P \mid R$$

$$R ::= 0 \mid R + R \mid \alpha.R \mid R|R \mid R \setminus H$$

### Proposição

*Se  $\Gamma$  é uma função parcial cujo contradomínio só tem expressões regulares então o  $\text{Reach}(\llbracket P \rrbracket_{\Gamma})$  é finito por estados para todo  $P \in \text{CCS}$  (i.e. o LTS atingível é finito por estados).*

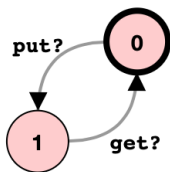
Este é o cálculo que se usa mais na prática.

- Calcular  $\llbracket Buffer | Buffer \rrbracket_{\Gamma}$ . para

$Buffer := put?.get?.Buffer$

- Calcular  $\llbracket Buffer | Buffer \rrbracket_{\Gamma}$ . para

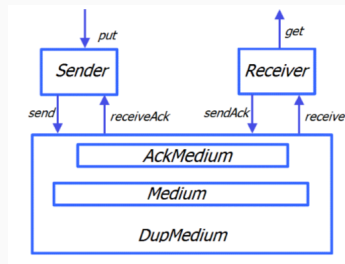
$Buffer := put?.get?.Buffer$



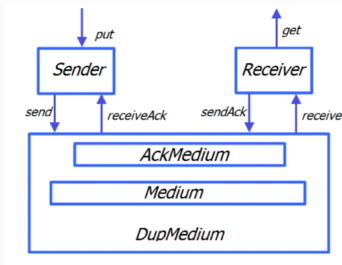
$BufferL := put?.pass!.BufferL$

$BufferR := pass?.get?.BufferR$

# Exemplo de um Protocolo com Ack (Pseuco)



## Exemplo de um Protocolo com Ack (Pseuco)



*Sender* := *put?.send!.Sending*

*Sending* := *receiveAck?.Sender*

*Receiver* := *receive?.get?.sendAck!.Receiver*

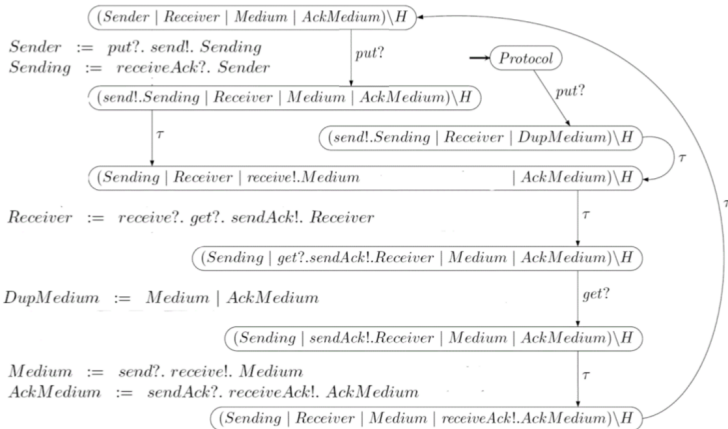
*Medium* := *send?.receive!.Medium*

*AckMedium* := *sendAck?.receiveAck!.AckMedium*

*DupMedium* := *Medium|AckMedium*

# Exemplo de um Protocolo com Ack (Pseuco, Holger H.)

$Protocol := (Sender \mid Receiver \mid DupMedium) \setminus \{send, receive, sendAck, receiveAck\}$



*Sender* := *put?.send!.Sending*

*Sending* := *receiveAck?.Sender + receiveNAck?.send!.Sending*

*Receiver* := *receive?.get?.sendAck!.Receiver +*  
*gargled?.sendNAck!.Receiver*

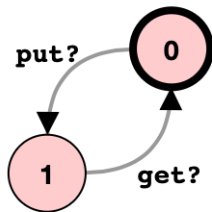
*Medium* := *send?.(receive!.Medium + i.garbled!.Medium)*

*AckMedium* := *sendAck?.receiveAck!.AckMedium +*  
*sendNAck?.receivedNAck!.AckMedium*

*DupMedium* := *Medium|AckMedium*

*Protocol* := (*Sender | Receiver | DupMedium*)\  
{*send, receive, sendAck, receiveAck,*  
*eNAck, sendNAck, gargled*}

Para um observador externo os processos *Buffer*, *Protocol* e *ProtocolG* têm o mesmo comportamento (mas LTSs diferentes, claro!).



[MISSING]

- **Secção Crítica:** porção de código que só pode ser executado por um processo num dado instante
- Supõe-se que a execução da secção crítica por um só processo termina.
- **MUTEX** o problema consiste em ter
  1. um algoritmo de entrada *acquire\_mutex()*
  2. um algoritmo de saída *release\_mutex()*
- Enquadrando a secção crítica garantem

**Exclusão mútua:** que o código da zona crítica é executado no máximo por um processo em cada instante.

**Starvation-freedom:** cada processo que invoca *acquire\_mutex()* termina, permitindo assim que os processos que querem entrar na zona crítica o possam fazer.

Se se modelar a entrada e a saída da zona crítica fica (para dois processos).

$$\textit{MutexSpec} \quad := \quad \textit{enter1.exit1.MutexSpec} + \textit{enter2.exit2.MutexSpec}$$

- $P_1, P_2$  processos
- variáveis partilhadas  $b_1, b_2$  e  $k$ , sendo
- se  $k = i$  então  $P_i$  pode entrar
- $P_1$  faz  $k = 2$  (dá o privilégio a  $P_2$ )
- e simetricamente para  $P_2$
- $b_i = \text{true}$  quando  $P_i$  espera
- $b_i = \text{false}$  quando  $P_i$  sai da zona crítica.

Processo  $P_i$

**while true do**

**noncritical actions**

$b_i \leftarrow \text{true};$

$k \leftarrow j;$

**while  $b_j \wedge k = j$  do**

**skip;**

**critical actions**

$b_i \leftarrow \text{false};$

O CCS não tem variáveis, mas estas são processos que comunicam a outros processos que necessitam de ler ou escrever os seus valores. Isto é

- As variáveis são **processos** cujos estados são os seus possíveis valores
- Para  $b_1$  temos estado  $B_{1t}$  se o seu valor for **true** e o estado  $B_{1f}$  se o seu valor for **false**.
- Outros processos podem ler ou escrever o valor de variáveis comunicando com o respectivo processo o valor pretendido
- Para um processo ler **true** em  $b_1$  sincroniza com esse processo por uma ação (canal)  $b1rt$
- Para um processo escrever **false** em  $b_1$  sincroniza com esse processo por uma ação (canal)  $b1wf$
- Análogamente se define o comportamento de  $k$  que pode tomar os valores 1 e 2.

$$B_{1f} := b1rf!.B_{1f} + b1wf?.B_{1f} + b1wt?.B_{1t}$$

$$B_{1t} := b1rt!.B_{1t} + b1wf?.B_{1f} + b1wt?.B_{1t}$$

$$B_{1f} := b1rf!.B_{1f} + b1wf?.B_{1f} + b1wt?.B_{1t}$$

$$B_{1t} := b1rt!.B_{1t} + b1wf?.B_{1f} + b1wt?.B_{1t}$$

$$B_{2f} := b2rf!.B_{2f} + b2wf?.B_{2f} + b2wt?.B_{2t}$$

$$B_{2t} := b2rt!.B_{2t} + b2wf?.B_{2f} + b2wt?.B_{2t}$$

$$B_{1f} := b1rf!.B_{1f} + b1wf?.B_{1f} + b1wt?.B_{1t}$$

$$B_{1t} := b1rt!.B_{1t} + b1wf?.B_{1f} + b1wt?.B_{1t}$$

$$B_{2f} := b2rf!.B_{2f} + b2wf?.B_{2f} + b2wt?.B_{2t}$$

$$B_{2t} := b2rt!.B_{2t} + b2wf?.B_{2f} + b2wt?.B_{2t}$$

$$K_1 := kr1!.K_1 + kw1?.K_1 + kw2?.K_2$$

$$K_2 := kr2!.K_2 + kw1?.K_1 + kw2?.K_2$$

- apenas modelar a entrada e saída da zona crítica
- fazer a inicialização das variáveis  $b_i$  e  $k$
- supomos que não podem terminar na zona crítica ou ficar lá para sempre
- para representar o ciclo **while** para  $P_1$  temos um estado  $P_{11}$  tal que
  - ler os valores de  $b_2$  e  $k$
  - esperar se  $b_2 = \text{true} \wedge k = 2$
  - mudar de estado,  $P_{12}$
  - em  $P_{12}$  entrar e sair da zona critica
  - mas como avaliar  $b_j \wedge k = j$ ?
  - avaliamos da esquerda para a direita: se  $b_j$  é **true** então avaliamos se  $k = j$ ; a segunda não é avaliada se a primeira for falsa.

$$P_1 := b1wt!.kw2!.P_{11}$$
$$P_{11} := b2rf?.P_{12} + b2rt?.(kr2?.P_{11} + kr1?.P_{12})$$
$$P_{12} := enter1.exit1.b1wf!.P_1$$

$$P_1 := b1wt!.kw2!.P_{11}$$

$$P_{11} := b2rf?.P_{12} + b2rt?.(kr2?.P_{11} + kr1?.P_{12})$$

$$P_{12} := enter1.exit1.b1wf!.P_1$$

$$P_2 := b2wt!.kw1!.P_{21}$$

$$P_{21} := b1rf?.P_{22} + b1rt?.(kr1?.P_{21} + kr2?.P_{22})$$

$$P_{22} := enter2.exit2.b2wf!.P_2$$

Se  $k = 1$  no início

$$Peterson := (P_1 | P_2 | B_{1f} | B_{2f} | K_1) \setminus L$$

onde  $L$  são todas as ações excepto as de entrada e saída na zona critica.

Ficheiro no Pseuco.com:

<https://pseuco.com/#/edit/remote/tkxz8fpu1t8ke56bj9uo>

Será que *MutexSpec* e *Peterson* são equivalentes? Mas para que equivalência?  
Interessa-nos o comportamento observável...

Contudo podemos mostrar que no algoritmo de Peterson um processo não entra na zona crítica se o outro lá está. Para tal usamos um monitor, i.e., outro processo que executando em paralelo irá detectar se há um erro:

$$MutualTest := enter1!.MutualTest1 + enter2!.MutualTest2$$
$$MutualTest1 := exit1!.MutalTest + enter2!.bad!.0$$
$$MutualTest2 := exit2!.MutalTest + enter2!.bad!.0$$

Executando

$$(Peterson|MutualTest)\{\mathit{enter1}, \mathit{enter2}, \mathit{exit1}, \mathit{exit2}\}$$

ver se a ação *bad!* pode ser executada. Pode-se mostrar que não. Ficheiro no

Pseuco.com: `tex`